

Vrije Universiteit Amsterdam



Forward Football



Bachelor Thesis

Detecting Pass-Containing Windows in Football Using Machine Learning

Author: Sami Finkbeiner (2797796)

1st supervisor: Wan Fokkink
daily supervisor: Ryan Stepfner (Forward Football)
2nd reader: Erik Link

*A thesis submitted in fulfillment of the requirements for
the VU Bachelor of Science degree in Computer Science*

June 2, 2026

Abstract

Raw positional tracking data provides detailed information about player and ball movement in football, but it does not directly identify discrete events such as passes. This study investigates whether machine learning models can predict pass-containing windows from raw positional data. The task is formulated as a supervised binary classification problem, where each non-overlapping 15 frame window, corresponding to 3 seconds of gameplay, is labelled according to whether it contains at least one pass.

Tracking data from 51 football matches was preprocessed, transformed into frame-level features, and aggregated into window-level representations describing ball movement and player proximity. Logistic Regression, Decision Tree, Random Forest, XGBoost, and a Multi-Layer Perceptron were trained and compared using group based cross-validation at the match level. Classification thresholds were selected on a separate validation set.

The results show that pass-containing windows can be predicted from engineered features derived from raw positional data. XGBoost achieved the strongest test performance, with an F1-score of 0.74 for both the pass and no-pass classes. Random Forest and the MLP performed competitively, while Logistic Regression and Decision Tree showed weaker overall performance. Feature importance analysis indicated that the most informative features described ball-player distance, changes in the closest player, ball movement path, direction, and field position.

These findings suggest that non-linear models are well suited to capturing interaction patterns involved in pass prediction. However, the results should be interpreted as probabilistic window-level predictions rather than complete event-level pass annotations, since the exact timing and number of passes within a window are not directly predicted.

Keywords: football analytics, pass detection, positional tracking data, machine learning, event detection, binary classification, feature engineering

Contents

1	Introduction	5
1.1	Problem Formulation	6
2	Related Work	6
3	Methodology	7
3.1	Data Description	7
3.2	Data Preprocessing	8
3.2.1	NaN-based Window Filtering	9
3.2.2	Train, Validation, and Test Split	9
3.3	Feature Engineering	10
3.3.1	Frame-level Feature Construction	10
3.3.2	Window-level Feature Construction	11
3.3.3	Feature Definitions and Derived Interaction Features	11
3.3.4	Feature Selection	13
3.4	Models	13
3.4.1	Model Pipeline	13
3.4.2	Cross-Validation Strategy	13
3.4.3	Class Imbalance	14
3.4.4	Hyperparameter Tuning	14
3.4.5	Threshold Selection	15
3.4.6	Evaluation Metrics	15
3.4.7	Final Model Training	15
3.5	Postprocessing of Predictions	16
3.6	Hardware and Implementation Details	16
3.7	Code Availability	17
4	Results	17
4.1	Confusion Matrix Analysis	18
4.2	Feature Importance	19
5	Discussion	20
5.1	Most Useful Features for Pass Prediction	20
5.2	Model Performance on the Pass Prediction Task	22
5.3	Overfitting and Generalization	22
5.4	Limitations	25
5.5	Future Work	26
6	Conclusion	27
A	Features	32
A.1	Final Feature Set	32
A.2	Feature Importance	33

B	Hyperparameters	35
B.1	Hyperparameter Search Space	35
B.2	Selected Hyperparameters	35
C	Postprocessing	37
C.1	Pass Compression	37
C.2	Proposed Postprocessing Algorithm	37
D	Validation Results	38

1 Introduction

Raw positional data has become increasingly important in football analytics in recent years [31]. By capturing the spatial positions of players and the ball over time, this type of data provides a detailed representation of match dynamics [19]. However, while positional data allows matches to be reconstructed, it does not directly provide discrete football events such as passes, shots, or tackles. Therefore, these events must be extracted from the tracking data before they can be used for football analysis [3].

This study focuses on the detection of pass-related patterns from raw positional data. Passes are selected because they are among the most frequent and fundamental events in football. They play an important role in describing team possession, ball progression, tactical structure, and player decision-making [19]. However, detecting passes from tracking data is challenging because passes vary in distance, speed, direction, and surrounding player context [3].

Rather than predicting the exact frame at which a pass occurs, this study uses a window-based formulation. The objective is to predict whether a fixed-length time window contains at least one pass. This provides a practical intermediate step between tracking data and event-level pass detection. If pass-containing windows can be identified reliably, additional post-processing methods can then be used to extract or approximate individual pass events within positive windows.

Although previous research has explored football event detection using rule-based and machine learning approaches, less attention has been given to comparing multiple model types for detecting events from raw positional data [3]. The research question guiding this study is therefore:

Which machine learning models perform best at predicting pass-containing windows in football using raw positional data?

To answer this question, this study uses tracking data from 51 football matches provided by Forward Football. Frame-level features are constructed from ball and player coordinates and then aggregated into window-level representations. Several machine learning models are trained and compared, including Logistic Regression, Decision Trees, Random Forests, Multi-Layer Perceptrons, and XGBoost. Model performance is evaluated using precision, recall, and F1-score, with particular attention to the F1-score as the main metric for comparing models under class imbalance.

The contribution of this study is to provide a systematic comparison of machine learning models for detecting pass-containing windows from raw positional tracking data. In addition to comparing model performance, the study examines how engineered football-specific features contribute to pass prediction. By formulating the task at the window level, the study provides an intermediate step between tracking data and event-level pass annotation. It also highlights the limitations of this formulation, particularly the need for post-processing when moving from window-level predictions to precise event-level detection.

In doing so, the study aims to support future research on extracting meaningful football events from tracking data by identifying which model types and feature representations are most effective for this task.

1.1 Problem Formulation

The task is formulated as a supervised binary classification problem [9], where the objective is to predict whether a fixed-length time window contains at least one pass. Rather than detecting the exact frame at which a pass occurs, the model assigns each window to one of two classes: pass-containing window or no-pass window. In this study, a pass is defined operationally as an annotated pass event in the event data provided by Forward Football.

Passes are inherently temporal events, meaning that both the context leading up to the pass and the immediate aftermath may be relevant for accurate prediction. To capture this temporal structure, a window-based approach is adopted, where sequences of consecutive frames are grouped into fixed-length windows.

Each window is treated as a single instance for the classification task. A window is labelled as positive (1) if it contains at least one frame corresponding to a pass event, and negative (0) otherwise. This labelling strategy allows the model to learn patterns associated with the occurrence of passes within a short temporal context.

2 Related Work

Over the past decade, research in football analytics has grown rapidly, driven by the increasing availability of high-resolution tracking and event data. Analytics is now widely used to support performance analysis, scouting, and tactical decision-making [19]. One key area within this field is event detection, which aims to identify actions such as passes, shots, or tackles from raw data. Despite advances in automated methods, manual annotation remains widely used as a reference for evaluating event detection methods, highlighting both the importance and the difficulty of reliable event detection [17].

A central distinction in the literature is between event-based and tracking-based approaches. Event-based datasets contain manually labelled actions and are widely used for downstream tasks such as match analysis or outcome prediction. In contrast, tracking data provides continuous spatial information on players and the ball but does not include explicit event labels [19]. As a result, detecting events directly from tracking data remains a challenging problem, requiring methods that can infer meaningful patterns from spatiotemporal signals [3].

Several approaches have been proposed for football event detection and related tasks. A common approach is the use of rule-based systems, which encode domain knowledge into predefined conditions. Previous studies have reported strong performance for such methods, with F1-scores of up to 92% for selected event types [29, 28, 20, 15]. However, these methods can be complex to design and may not generalize well across different datasets, match contexts, or changes in playing style. Moreover, reported performance in football event detection may be difficult to compare across studies due to differences in preprocessing, validation procedures, and potential overfitting due to data leakage [3].

Machine learning approaches have also been applied to football event detection. Unlike rule-based methods, these approaches allow models to learn event-related patterns directly from data. Richly et al. investigated the recognition of compound events in spatio-temporal football data using machine learning methods, including Support Vector Machines, Decision Trees, and Random Forests [24]. They later used a neural network to detect kick and ball reception events, reporting F1-scores above 90% when data from different matches

was merged, but a lower F1-score of 81% when evaluating across separate matches [25]. This highlights the importance of evaluation strategy and supports the use of match-level evaluation when assessing generalization to unseen games.

Research focusing specifically on passing has often concentrated on evaluating pass quality rather than detecting pass events themselves. Sofroni [27] developed an *Expected Successful Pass* model using data provided by Forward Football, building on earlier work by Anzer and Bauer [1] and Boon [4]. Similarly, Horton et al. constructed spatiotemporal features based on computational geometry and applied machine learning to classify passes into qualitative categories, showing that complex spatial features can support the evaluation of passing actions [13]. El Kadi also addressed a pass-related detection task using Forward Football data by applying Random Forest and XGBoost to detect line-breaking passes [8].

More broadly, Herold et al. provide a comprehensive review of machine learning applications in professional football, highlighting the increasing use of tracking data and the potential of data-driven approaches, while also emphasizing limitations related to data quality, model interpretability, and practical implementation [12]. Additional studies have explored machine learning for match outcome prediction [11] and tactical analysis [1] using both event and tracking data, further demonstrating the versatility of these methods.

Despite these advances, existing reviews suggest that event detection from raw tracking data remains relatively underexplored compared with other football analytics tasks [3]. In particular, there is a lack of systematic comparisons across different model types to determine which approaches are best suited for this task when dealing with noisy and incomplete tracking data. Furthermore, while traditional machine learning methods are well represented in the literature, the application of neural networks to this specific problem appears to be less explored.

This study aims to address this gap by comparing multiple machine learning models for pass prediction using raw tracking data, providing insights into their relative performance and suitability for event detection tasks in football analytics.

3 Methodology

3.1 Data Description

The data used in this study was provided by Forward Football and consists of tracking data collected using a Local Positioning Measurement (LPM) system [23]. Six LPM sensors are installed around the pitch and track the spatial positions of both players and the ball throughout a match. The data was collected primarily from youth football matches.

For each match, three main data sources are available:

- An Excel file containing raw positional tracking data, including the x and y coordinates of the ball and all players over time.
- A CSV file containing timestamped pass events.
- An additional CSV file with overall match statistics, used to identify the team associated with each player.

The dataset comprises 51 matches and originally contained 33,869 annotated pass events. After NaN-based window filtering, as described in Section 3.2, 45 passes were removed, leaving 33,824 pass events in the final dataset. According to the dataset documentation provided by Forward Football, the tracking data was recorded at a sampling rate of 5 Hz, corresponding to one frame every 0.2 seconds. In total the dataset contains approximately 1,112,820 frames, corresponding to roughly 3,709 minutes of gameplay.

The raw data presents several challenges. First, missing values are present in both player and ball coordinates. Missing player positions are expected in some cases, for example, when players are off the field or temporarily not tracked. However, missing ball positions are more problematic, as the ball is central to the analysis.

Second, some coordinates fall outside the pitch dimensions (105×68 meters) used in the dataset, indicating measurement noise or tracking inaccuracies. In a small number of cases, pass events are also recorded outside these boundaries, suggesting potential synchronization or labeling inconsistencies.

These data quality issues require careful preprocessing, which is described in Section 3.2.

To better understand the spatial characteristics of pass events across the dataset, Figure 1 shows the normalized spatial distribution of pass events for the training, validation, and test sets.

The distributions appear consistent across all data splits, with similar spatial patterns observed throughout the pitch. This suggests that the training, validation, and test sets are representative of the same underlying data distribution, supporting the validity of the evaluation results.

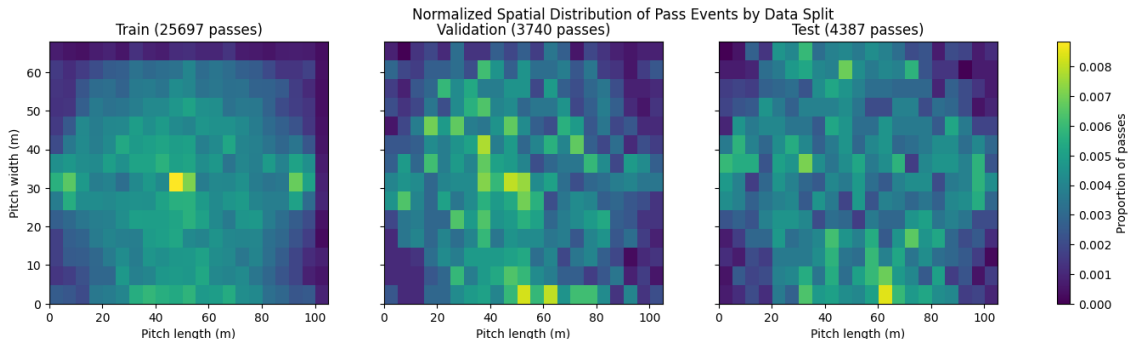


Figure 1: Normalized spatial distribution of pass events across the training, validation, and test sets.

3.2 Data Preprocessing

All numeric columns were converted to appropriate numeric data types to ensure consistency in subsequent processing steps. Duplicate timestamps in both the tracking and pass event data were identified and removed. Timestamps were then sorted for both datasets to ensure temporal consistency.

Following this, the tracking data and pass event data were merged into a single dataframe based on their timestamps. This allowed each frame to be labeled as either containing a pass event or not.

To address spatial inconsistencies, all positional coordinates were constrained within the pitch dimensions used in the dataset (105×68 meters). A tolerance of 5 meters was applied. Values slightly outside the pitch boundaries were clipped to the nearest valid boundary, while values exceeding this tolerance were considered erroneous and set to missing (NaN). This approach was chosen to retain slightly noisy but otherwise valid observations, while removing clearly invalid data points.

Missing values in both ball and player coordinates were handled using linear interpolation. Previous work on football tracking data has compared several interpolation techniques for estimating missing coordinates and found that linear interpolation performs reasonably well, compared to more advanced methods [16]. Interpolation was applied only for gaps of up to 5 consecutive frames (equivalent to 1 second), as longer gaps were considered too large to reliably reconstruct without introducing significant noise.

3.2.1 NaN-based Window Filtering

After feature extraction and window construction (see Section 3.3), an additional filtering step was applied to remove windows with a high proportion of missing values. Windows with extensive missing data were considered unreliable because many of their derived features would depend largely on imputed rather than observed values. This is particularly problematic for movement-based features such as ball speed, direction change, distance to the closest player, and player-change indicators, which can be distorted when too many coordinates are missing.

For this reason, windows containing more than 50% missing values were removed before model training. This threshold was chosen as a pragmatic data quality criterion: it allowed windows with mostly complete information to be retained, while excluding windows where the majority of the tracking information was missing. This reduced the risk that the models would learn patterns introduced by missing-data handling rather than genuine pass related movement patterns.

This filtering step removed 3,705 no-pass windows, corresponding to 8.22% of the negative-class windows, and 38 pass-containing windows, corresponding to 0.13% of the positive-class windows. The number of annotated pass events removed by this step is reported in Section 3.1.

The filtering step mainly affected the negative class, while its impact on the positive class was negligible. As a result, the procedure slightly reduced the class imbalance by removing a larger number of no-pass windows than pass-containing windows. More importantly, it improved the reliability of the training data by excluding windows where the feature values would be based on insufficient tracking information.

A small comparison experiment was also conducted with and without this filtering step. The results were similar, suggesting that the filtering did not substantially change the overall model performance. However, the filtered dataset was used in the final experiments because it provided a more controlled input representation and reduced the dependence on imputation in highly incomplete windows.

3.2.2 Train, Validation, and Test Split

Finally, the dataset was split into training, validation, and test sets at the match level using an 80/10/10 split. The dataset consisted of 51 matches, representing the full set of

Forward Football matches available for this study. Since 51 could not be divided evenly according to this ratio, the additional match was assigned to the test set. This resulted in 40 matches for training, 5 matches for validation, and 6 matches for testing. The split allowed most of the available data to be used for model training, while still retaining separate validation and test sets with a substantial number of positive pass instances. The final split contained 25,697 passes in the training set, 3,740 passes in the validation set, and 4,387 passes in the test set.

The use of 51 matches provides a relatively broad dataset compared with several related studies based on football tracking data. For example, Richly et al. [24] used a manually annotated gold standard consisting of 194 events from sections of two matches. Although manual annotation may provide reliable event labels, such a limited number of matches restricts the extent to which results can be generalized. Bischofberger et al. [3] used data from 27 matches, while Morra et al. [20] used a synthetic dataset generated from 8 simulated football matches, with 2,670 pass events in the training set and 2,389 pass events in the test set. Finally, Sofroni [27], who also used Forward Football data to study expected successful passes, used 28 matches. In comparison, using 51 matches in total, with 40 reserved for training, provides a broader basis for learning match-level variation while still allowing evaluation on unseen matches.

The split was performed at the match level to reduce the risk of overfitting and overly optimistic evaluation results. Events and tracking patterns within the same match are temporally and contextually related, meaning that windows from a single match are not fully independent. For example, structured game situations such as kick-offs, goal kicks, and corners often follow repeated movement patterns and are frequently associated with passes. If windows or frames from the same match were randomly assigned to both training and evaluation sets, the model could learn match-specific patterns rather than generalizable pass-related patterns. This would make the evaluation data too similar to the training data and could lead to inflated performance estimates [3].

A match-level split therefore provides a more realistic evaluation setting. Since the validation and test sets consist of complete matches that were not used for training, the model is evaluated on unseen match contexts. This reflects the intended use case, where a trained model should generalize to new matches rather than recognize patterns from matches already observed during training.

3.3 Feature Engineering

3.3.1 Frame-level Feature Construction

Feature engineering was first performed at the frame level using the raw positional coordinates of the ball and players. The purpose of this step was to describe the instantaneous state of the game at each recorded frame. Several of the frame-level features were inspired by previous work on event recognition in positional football data [24], where similar ball movement and player context features are described in more detail.

The frame-level features describe both ball movement and the local player context around the ball. Ball movement features included speed, acceleration, and changes in direction. Player-context features included the distance from the ball to the nearest player, the team of the closest player, and the number of attacking and defending players within a defined radius of the ball.

These features provide a frame-by-frame representation of the physical dynamics of the ball and its surrounding players. However, they were not used directly as model inputs. Instead, they were used as the basis for constructing the window-level features described in Section 3.3.2.

3.3.2 Window-level Feature Construction

The choice of window size and stride length is an important design decision because it affects the temporal context available to the model, the number of training instances, and the interpretation of the labels. After preliminary testing, a window size of 15 frames and a stride length of 15 frames were selected. Given the 5 Hz sampling rate of the tracking data, this corresponds to non-overlapping windows of 3 seconds of gameplay.

A 3 second window was chosen because it provides a short but meaningful temporal context around a possible pass. Passes are not only defined by the frame at which they are annotated, but also by the movement patterns immediately before and after the event, such as changes in ball speed, direction, distance to the closest player, and changes in the closest player. A very short window would provide less information about these temporal dynamics, while a much longer window could include several unrelated actions and make the label less specific.

Non-overlapping windows were chosen to keep the classification task interpretable and computationally manageable. With overlapping windows, the same pass could appear in many neighbouring windows, increasing the number of highly similar training examples and requiring additional post-processing to deal with duplicate detections. Non-overlapping windows therefore provide a clearer formulation: each segment of gameplay is represented once, and each window is labelled according to whether it contains at least one pass.

This design choice affects the results in several ways. First, it reduces the frame-level class imbalance by grouping frames into larger temporal units. Second, it means that the model predicts pass-containing windows rather than exact pass frames. Third, because each window covers 3 seconds, multiple passes can occasionally occur within the same positive window. Therefore, the reported results should be interpreted as window-level classification performance rather than event-level pass detection performance. The limitations of this design choice are further discussed in Section 5.4.

For each window, frame-level features were aggregated into a single window-level representation. This was done by computing descriptive statistics within each window, such as the mean, maximum, minimum, and standard deviation of selected frame-level features. These statistics describe how the feature values changed during the window, rather than representing uncertainty in the model results. In addition, some features were extracted from specific positions within the window, such as the first frame, middle frame, and final frame.

The resulting window-level features (see Appendix A.1) formed the main input variables used for model training.

3.3.3 Feature Definitions and Derived Interaction Features

Several feature names use the terms *start*, *center*, and *end*. These refer to the first frame, middle frame, and final frame of the 15 frame window, respectively. For example, `dist_`

start denotes the distance between the ball and the closest player at the first frame of the window, while **dist_end** denotes the same distance at the final frame. Features such as **speed_end_minus_start** describe the change in a feature value between the start and end of the window.

The term *pitch origin* refers to the fixed coordinate origin of the pitch coordinate system. The feature **ball_displacement_from_pitch_origin** measures the distance of the ball from this fixed reference point. This provides a compact representation of the ball’s pitch location by combining information from both the x and y coordinates.

Several ball trajectory features were also included. The feature **ball_path_length** represents the total distance travelled by the ball within the window, while **ball_straight_distance** represents the direct distance between the ball’s start and end positions. The difference between these values is captured by **ball_curve_distance**, which describes how much the ball trajectory deviates from a straight line.

In some derived feature names, **x** denotes multiplication rather than the pitch coordinate. These features are interaction terms that combine two variables to capture joint patterns. For example, **player_changed_x_path_length** multiplies an indicator of whether the closest player changed by the ball path length. This captures cases where a player change occurs together with substantial ball movement, which may be indicative of a pass.

Additional derived interaction features were included to describe combinations of ball movement and player proximity. The feature **player_changed_and_close_edges** is set to 1 when the closest player to the ball changes between the start and end of the window, and the ball is within 2 meters of the nearest player at both the start and end. This was designed to capture situations where the ball moves from being close to one player to being close to another.

The feature **player_changed_and_high_speed** is set to 1 when the closest player changes during the window and the maximum ball speed reaches at least 5 m/s. This combines a possible change in ball possession or proximity with strong ball movement. The motivation for this feature is consistent with previous work on football event detection, where ball movement and player-ball vicinity are used as indicators for detecting pass-like actions [3].

Similarly, **high_speed_far_from_player_share** and **high_speed_close_to_player_share** describe the proportion of frames in the window where the ball speed is at least 5 m/s while the ball is either far from the nearest player, defined as more than 3 meters, or close to the nearest player, defined as at most 2 meters. These features help distinguish between fast ball movement in open space and fast ball movement near players.

Ideally, the thresholds used in these derived features would be treated as hyperparameters rather than fixed choices. However, tuning these thresholds would substantially increase the feature engineering search space and make the comparison between models less controlled. Since the main objective of this study was to compare different machine learning models using a common feature representation, fixed thresholds were used to keep the input features consistent across all models.

This means that the reported results partly depend on the selected threshold values. Different values could lead to different feature representations and may therefore affect model performance. However, these thresholds were used to encode general football domain assumptions, rather than to directly classify passes. They should therefore be interpreted as part of the feature design, not as optimized event-detection rules.

Limitations of threshold design decisions are further discussed in Section 5.4

3.3.4 Feature Selection

After constructing the window-level feature set, feature selection was performed to reduce redundancy, improve model efficiency, and remove features that did not contribute meaningfully to prediction performance.

Feature relevance was assessed on the validation set using permutation importance. The values of each feature are randomly permuted and the resulting decrease in model performance is used as an estimate of that feature’s contribution to prediction [5].

Feature selection also served to reduce the risk of overfitting. A large number of engineered features can increase the chance that models learn noise or dataset specific patterns rather than general pass-related signals. Features with consistently low or negative permutation importance were therefore removed from the final feature set. This reduced the dimensionality of the input representation, lowered computational cost, and helped focus the models on features that contributed more consistently to validation performance.

A complete list of the final features used for model training is provided in Appendix A.1.

3.4 Models

3.4.1 Model Pipeline

Several machine learning models were evaluated in this study, including Logistic Regression [7], Decision Trees [22], Random Forests [5], XGBoost [6], and a Multi-Layer Perceptron (MLP) [26].

Each model was implemented within a structured pipeline to ensure consistency and reproducibility across experiments. The pipeline included an imputation step to handle any remaining missing values after preprocessing. The imputation strategy was treated as a hyperparameter, with both mean and median imputation considered.

For models sensitive to feature scaling, such as Logistic Regression and the MLP, an additional standardization step was included in the pipeline. This ensured that features were placed on a comparable scale, which is important for models that rely on distance, gradients, or weight optimization. Tree-based models, such as Decision Trees, Random Forests, and XGBoost, do not require feature scaling in the same way, since they split features based on threshold values.

3.4.2 Cross-Validation Strategy

Model evaluation and hyperparameter tuning were performed using cross-validation on the training set. A group based cross-validation strategy was adopted, where each match was treated as a separate group. This ensured that windows from the same match did not appear in both the training and validation folds. As a result, each validation fold consisted of complete matches that were not used for training in that fold, reducing the risk of overfitting and providing a more realistic estimate of model generalization.

A GroupKFold [21] cross-validation strategy was used during hyperparameter tuning on the training set. In each fold, complete matches were kept together, so that all windows from the same match were assigned either to the training portion or to the internal validation portion of that fold, but never to both. This process was repeated across five folds.

3.4.3 Class Imbalance

Table 1 shows the overall class balance across all matches after NaN-based window removal. The final dataset contains 70,448 windows, of which 29,086 are pass-containing windows and 41,362 are no-pass windows. This corresponds to 41.29% positive windows and 58.71% negative windows.

Table 1: Overall window-level class distribution after NaN-based window removal

Class	Number of windows	Percentage (%)
0 (No-pass window)	41,362	58.71
1 (Pass-containing window)	29,086	41.29

Table 2 shows the class distribution separately for the training, validation, and test sets. The proportion of pass-containing windows is similar across the three splits, indicating that the match-level split preserves a broadly comparable class balance.

Table 2: Window-level class distribution by train, validation, and test split

Split	Total windows	Pass windows	Pass windows (%)
Training	53,888	22,103	41.02
Validation	7,779	3,226	41.47
Test	8,781	3,757	42.79

Although the window-based formulation reduced the class imbalance compared with frame-level classification, the dataset still contained more no-pass windows than pass-containing windows.

For the models where imbalance handling was supported, different weighting strategies were considered during hyperparameter tuning. These included no additional weighting, automatically balanced weighting, and manually specified weights. The manually specified weights were based on the ratio between the number of no-pass and pass-containing windows in the training set. This ratio was calculated as:

$$s = \frac{31,785}{22,103} \approx 1.438$$

The value s was therefore included as a candidate setting to increase the relative importance of the pass-containing class during training.

The purpose of these weighting strategies was to reduce the tendency of the models to favour the majority class. By assigning a higher penalty to errors on the minority class, the models were encouraged to pay more attention to pass-containing windows instead of primarily optimizing performance on no-pass windows.

For the MLP, class weighting was not included in the final hyperparameter search, so imbalance was addressed through threshold selection.

3.4.4 Hyperparameter Tuning

Hyperparameter optimization was conducted using randomized search with cross-validation [2]. For each model, 50 hyperparameter combinations were sampled and evaluated using the

five-fold GroupKFold strategy described above. Randomized search was preferred over grid search because it reduces computational cost while still allowing a broad exploration of the hyperparameter space [2].

The hyperparameter search space was defined separately for each model and included settings related to model complexity, regularization, learning behaviour, missing-value imputation, and class-imbalance handling where applicable.

Parameters such as maximum tree depth, minimum samples per leaf, regularization strength, learning rate, and neural network architecture were carefully considered to counteract overfitting. These parameters influence the extent to which a model can fit patterns in the training data. By selecting hyperparameters through group-based cross-validation, the aim was to identify configurations that generalized across unseen validation matches instead of configurations that only performed well on the training matches.

To ensure a fair comparison, all models were evaluated using the same cross-validation strategy and optimization metric. A summary of the hyperparameter search space for each model is provided in Appendix B.1.

3.4.5 Threshold Selection

Most classification models output a probability or confidence score for each window. To convert these scores into binary predictions, a classification threshold must be selected. The default threshold is commonly set to 0.5, meaning that windows with predicted probabilities greater than or equal to 0.5 are classified as pass-containing windows, while the remaining windows are classified as no-pass windows.

In this study, the classification threshold was selected using the validation set rather than fixed at 0.5. Different threshold values were evaluated, and the threshold that produced the highest F1-score on the validation set was selected. This was done because the optimal balance between precision and recall may not occur at the default threshold, particularly in an imbalanced classification setting.

The selected threshold was then kept fixed and applied to the test set during final evaluation.

3.4.6 Evaluation Metrics

Model performance was evaluated using precision, recall, and F1-score for both classes [9]. Precision measures the proportion of predicted positive windows that were correct, while recall measures the proportion of actual positive windows that were identified. The F1-score is the harmonic mean of precision and recall and was used as the main comparison metric because it balances false positives and false negatives in the presence of class imbalance. Accuracy was not used as the primary metric because it can be misleading in imbalanced classification problems, where a model may achieve high accuracy by favoring the majority class.

3.4.7 Final Model Training

After hyperparameter tuning and threshold selection, the best-performing configuration for each model was selected. The selected model was then retrained using the combined

training and validation sets. This allowed the final model to make use of all available development data before being evaluated on the test set.

The test set was held out during model development and used only once for final evaluation. It was not used for hyperparameter tuning, threshold selection, or model selection, ensuring an unbiased estimate of performance on unseen matches.

3.5 Postprocessing of Predictions

A naive way to interpret a positive window-level prediction is to assume that each predicted positive window corresponds to one pass event, with the center frame of the window used as the estimated pass frame. However, this assumption is not always valid. With a window size of 15 frames and a stride length of 15 frames, a single window can contain more than one pass. As shown in Table 3, windows containing three or more passes are rare, but windows containing two passes occur often enough to be relevant. Therefore, directly treating each positive window as one event may underestimate the true number of passes.

Table 3: Distribution of real passes in positive windows

Passes per window	Number of windows	Positive windows (%)
1	24,684	84.87
2	4,079	14.02
3	311	1.07
4	11	0.04
5	1	0.00

In this study, the models are evaluated at the window level. As a result, the reported results indicate how well the models identify whether a window contains at least one pass, rather than whether they precisely localize every individual pass event. Converting window-level predictions into event-level detections would require an additional postprocessing step.

One possible approach is to train a secondary count classifier that predicts whether a positive window contains zero, one, or two or more passes. This could then be combined with the binary pass classifier to estimate the number and approximate location of pass events within each positive window. A detailed summary of the pass compression caused by window-level labelling, together with a conceptual outline of this proposed postprocessing approach, is provided in Appendix C.1.

This post-processing approach was not implemented or evaluated as part of the final experiments, but is included as a possible direction for future work.

3.6 Hardware and Implementation Details

All experiments were conducted on a machine equipped with a 12th Gen Intel Core i7-12650H processor (10 cores, 16 logical processors) and 16 GB of RAM.

The models and evaluation pipeline were implemented in Python using the `scikit-learn` library [21]. Data processing and analysis were performed using `pandas` [18] and `NumPy` [10], while visualizations were generated using `matplotlib` [14] and `seaborn` [30].

3.7 Code Availability

To support reproducibility, the code used for preprocessing, feature extraction, model training, evaluation, and visualization is available in the following GitHub repository:

<https://github.com/zitherean/bsc-thesis-pass-detection.git>

4 Results

Table 7 in Appendix B.1 presents the selected hyperparameter configuration and classification threshold for each model.

The selected configurations show both similarities and differences across models. Mean imputation was selected for most models, suggesting that it was generally more suitable than median imputation for the final feature set. The ensemble-based models, Random Forest and XGBoost, both selected 300 estimators, indicating that they benefited from a larger number of trees. However, the models differed in their preferred complexity and imbalance-related settings. For example, the tree-based models selected different maximum depths and splitting criteria, while XGBoost selected a relatively shallow tree depth combined with regularization. The MLP selected a simple architecture with a single hidden layer of 64 neurons, suggesting that a more complex neural network did not improve validation performance in this setting.

Table 4: Model performance on the test set

Model	Prec (0)	Rec (0)	F1 (0)	Prec (1)	Rec (1)	F1 (1)
Logistic Regression	0.83	0.60	0.69	0.61	0.84	0.70
Decision Tree	0.86	0.55	0.68	0.60	0.88	0.71
Random Forest	0.86	0.62	0.72	0.63	0.86	0.73
XGBoost	0.87	0.64	0.74	0.64	0.87	0.74
MLP	0.86	0.62	0.72	0.63	0.87	0.73

Table 4 shows the performance of the evaluated models on the test set. Overall, XGBoost achieves the strongest performance, obtaining the highest F1-score for both the no-pass class and the pass class. Random Forest and MLP perform similarly, while Logistic Regression and Decision Tree obtain slightly lower overall performance. The corresponding validation set results are provided in Appendix D.

Across all models, recall for the pass class is relatively high, ranging from 0.84 to 0.88. This indicates that the models are generally effective at identifying windows that contain at least one pass. However, precision for the pass class is lower, ranging from 0.60 to 0.64. This shows that a considerable number of no-pass windows are incorrectly classified as pass windows.

The opposite pattern can be observed for the no-pass class. Precision for class 0 is high, ranging from 0.83 to 0.87, while recall is lower, ranging from 0.55 to 0.64. This indicates that when the models predict a no-pass window, this prediction is usually reliable, but many actual no-pass windows are still misclassified as pass windows.

This asymmetry suggests that the models tend to favor detecting pass windows, resulting in high recall for class 1 but a relatively large number of false positives. This behavior

is consistent with the objective of maximizing F1-score for pass detection, but it also shows that distinguishing pass-like movement patterns from actual pass windows remains challenging.

4.1 Confusion Matrix Analysis

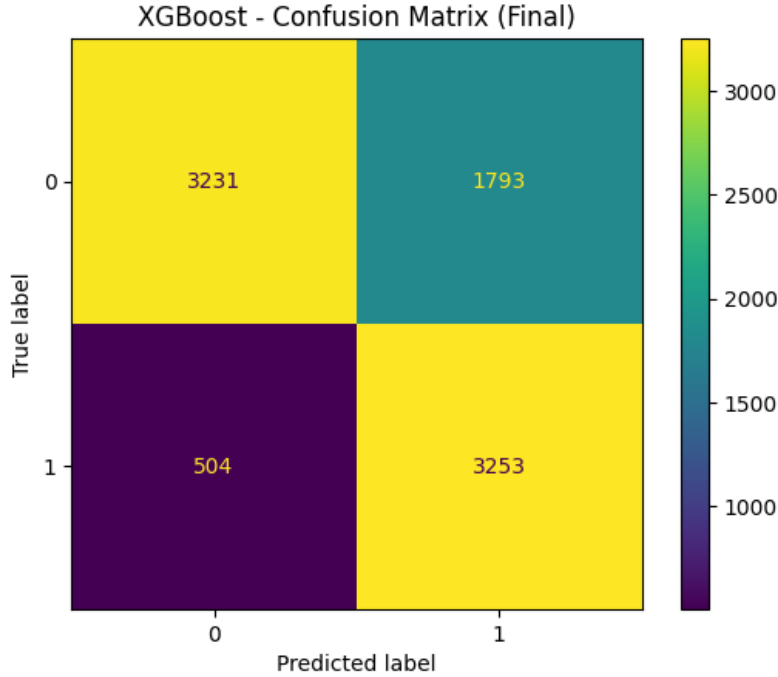


Figure 2: Confusion matrix of the XGBoost model on the test set

As shown in Figure 2, the XGBoost model correctly classifies 3,231 no-pass windows and 3,253 pass windows. At the same time, 1,793 no-pass windows are incorrectly classified as pass windows, while 504 pass windows are incorrectly classified as no-pass windows.

These results confirm that XGBoost is highly effective at detecting pass windows, achieving a high recall for class 1. Most windows containing at least one pass are correctly identified, and the number of false negatives remains relatively low. However, the model is less effective at identifying no-pass windows, as shown by the larger number of false positives. This indicates that the model tends to overpredict the pass class.

This pattern is consistent with the results presented in Table 4. While XGBoost achieves the highest overall performance among the evaluated models, its predictions still reflect a trade-off between detecting as many pass windows as possible and avoiding false alarms. In practice, the model favors recall for pass detection, which is beneficial for identifying pass events but leads to some confusion between true pass windows and no-pass windows with similar movement patterns.

4.2 Feature Importance

Permutation importance was used to analyze which engineered features contributed most strongly to model performance. This analysis was conducted on the validation set and was used during the feature refinement process. Features with consistently low or negative permutation importance were removed, while features that contributed positively to model performance were retained.

XGBoost was the best-performing model, and its feature importance scores are shown in Figure 3. The five highest-ranked features were:

- `player_changed_x_path_length`
- `dist_end`
- `dist_end_minus_start`
- `direction_min`
- `x_center`

These features show that the model relied on a combination of player-ball interaction, distance to the closest player, ball movement direction, and field position.

Overall, the feature importance results suggest that pass-window detection depends not only on the speed and trajectory of the ball, but also on how the ball’s movement relates to nearby players. This supports the use of engineered interaction features in addition to basic ball movement features.

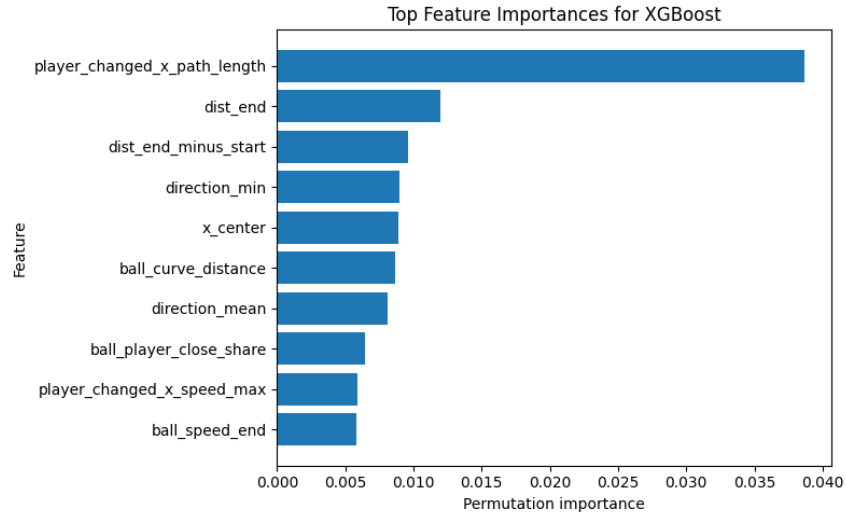


Figure 3: Top permutation importance scores for the XGBoost model on the validation set

The feature importance results for the remaining models are presented in Appendix A.2.

5 Discussion

5.1 Most Useful Features for Pass Prediction

The feature importance results suggest that the most useful features are not the raw coordinates alone, but engineered features that describe the interaction between the ball, nearby players, and ball movement over time.

Across the evaluated models, several feature groups appeared repeatedly among the most important predictors. These included:

- Distance-based features:
 - `dist_end`
 - `dist_end_minus_start`
 - `dist_mean`
 - `dist_start`
- Ball trajectory features:
 - `ball_path_length`
 - `ball_curve_distance`
 - `ball_displacement_from_pitch_origin`
- Interaction features:
 - `player_changed_x_path_length`
 - `player_changed_and_high_speed`
 - `ball_player_close_share`

This indicates that pass prediction depends on both the physical movement of the ball and its relationship to surrounding players.

For the tree-based models, especially Decision Tree and XGBoost, the most important feature was:

- `player_changed_x_path_length`

This feature combines information about whether the closest player to the ball changes during the window with the length of the ball’s movement path. Its importance is meaningful because a pass often involves the ball moving away from one player and towards another. Therefore, changes in the closest player, combined with meaningful ball movement, provide a strong signal for identifying pass windows.

The Random Forest model relied more strongly on distance and speed-change features. Its highest-ranked features were:

- `dist_end_minus_start`
- `dist_start`

- `speed_end_minus_start`
- `dist_player_change_mean`
- `dist_mean`

This suggests that Random Forest captured patterns related to how the distance between the ball and nearby players changes throughout a window. Such features are relevant because passes often involve a transition in ball possession or ball proximity, where the ball moves from being close to one player to becoming close to another.

For Logistic Regression, the most important features were:

- `player_changed_and_high_speed`
- `ball_displacement_from_pitch_origin`
- `dist_mean`
- `dist_max`
- `x_center`

Since Logistic Regression is a linear model, it appears to benefit from features that encode relatively direct relationships with the target, such as ball movement intensity, average distance to players, and field position. The importance of `x_center` also suggests that the location of the ball on the pitch contributes to pass prediction, possibly because some areas of the pitch contain more structured passing patterns than others.

For the MLP model, the highest-ranked features were:

- `ball_path_length`
- `ball_displacement_from_pitch_origin`
- `direction_mean`
- `streak_start`
- `x_center`

This indicates that the MLP relied strongly on ball trajectory and movement direction, as well as features describing the continuity of the closest player. Compared with the tree-based models, the MLP placed more emphasis on movement and trajectory features, which may reflect its ability to combine scaled continuous variables in a more flexible way.

Overall, the most useful features were those that transformed raw positional coordinates into meaningful descriptions of game dynamics. Raw x- and y-coordinates alone do not explicitly describe whether the ball is being passed. This supports the importance of feature engineering when using raw positional data for pass prediction.

5.2 Model Performance on the Pass Prediction Task

Overall, XGBoost achieved the best performance on the test set, with an F1-score of 0.74 for both class 0 and class 1. Random Forest and MLP followed closely, while Logistic Regression and Decision Tree showed slightly weaker performance.

The results suggest that non-linear models are better suited to this task than the linear baseline. This is consistent with the ability of tree-based ensemble methods to capture non-linear relationships and feature interactions [5, 6]. XGBoost and Random Forest are able to capture complex relationships between engineered features, such as ball movement, distance to nearby players, and changes in the closest player. These interactions are important for pass prediction, because a pass is not defined by ball movement alone, but by how the ball moves in relation to players.

The Decision Tree model achieved the highest recall for the pass class, but this came at the cost of lower recall for the no-pass class. This indicates that the model tended to overpredict pass windows. Logistic Regression achieved the lowest overall performance, likely because the relationship between positional features and pass events is not purely linear.

The MLP model also performed strongly, achieving an F1-score of 0.73 for the pass class. This shows that the neural network was able to learn useful patterns from the engineered features. However, it did not outperform XGBoost, suggesting that the available dataset and feature set were particularly well suited to tree-based ensemble methods. This may be because tree-based models can effectively capture non-linear relationships and feature interactions without requiring the same level of data volume or model complexity as neural networks.

Although the best-performing model achieved reasonable performance, the results also show that pass prediction from raw positional data remains a challenging task. The differences between models suggest that engineered features provide meaningful predictive information, but that the task is still affected by ambiguity at the window level. In particular, some pass and no-pass windows may contain similar movement patterns, making perfect separation difficult.

This reinforces the importance of evaluating models on unseen matches, since event-detection performance can be strongly affected by the validation strategy [25, 3].

The results show that XGBoost was the most effective model for this task, but the close performance of Random Forest and MLP indicates that the predictive signal is present across different non-linear approaches. The weaker performance of Logistic Regression highlights the importance of modelling feature interactions when predicting passes from positional data.

5.3 Overfitting and Generalization

Overfitting is an important risk in this study because the dataset consists of many window-level samples derived from a relatively limited number of matches. Although the final dataset contains 70,448 windows, these windows are not fully independent, since many of them come from the same matches and share temporal, tactical, and tracking-related characteristics. Therefore, the effective amount of independent data is smaller than the total number of windows suggests. A larger and more diverse dataset would provide a stronger basis for evaluating generalization.

Several methodological choices were made to reduce this risk. First, the train, validation, and test split was performed at the match level, ensuring that windows from the same match did not appear in multiple splits. Second, hyperparameter tuning was performed using GroupKFold cross-validation, where each fold validated the model on complete unseen matches. Third, model complexity was controlled through hyperparameters such as tree depth, regularization strength, minimum samples per leaf, and neural network architecture. Finally, feature selection was used to remove features that did not contribute consistently to validation performance.

The validation and test results suggest that the countermeasures were useful. Figure 4 shows the mean validation F1-score for each model across the five GroupKFold cross-validation folds. The error bars represent one standard deviation around the mean, indicating how much the F1-score varied across the different validation folds.

The small error bars suggest that the models achieved relatively stable validation performance across different match-level splits. This indicates that performance was not strongly dependent on one specific validation fold, but remained similar when different groups of matches were used for validation during cross-validation. XGBoost achieved the highest mean validation F1-score, followed closely by Random Forest. This supports the use of match level cross-validation to reduce overfitting.

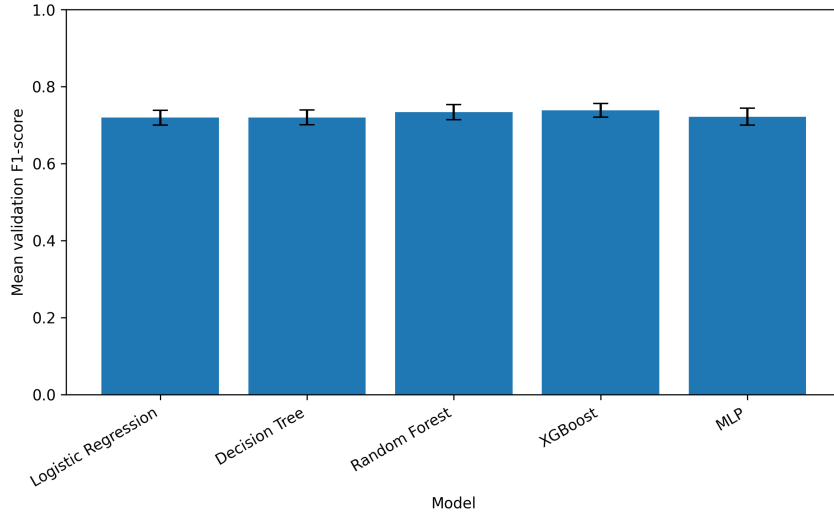


Figure 4: Mean validation F1-score with standard deviation across five GroupKFold cross-validation folds.

Figure 5 compares the F1-score for the pass-containing class on the validation and test sets, providing an indication of how stable each model’s performance remained on unseen matches. Across all models, the F1-scores were only slightly lower on the test set than on the validation set. For the pass class, the decrease ranged from 0.01 to 0.03, while for the no-pass class the decrease ranged from 0.04 to 0.06. This indicates that performance dropped moderately on unseen test matches, but not enough to suggest severe overfitting. The largest difference was observed for the Decision Tree model in recall for the no-pass class, which decreased from 0.64 on the validation set to 0.55 on the test set. This suggests that the Decision Tree was less stable than the ensemble-based models. In contrast,

the XGBoost recall for the pass class remained unchanged between validation and test, indicating stable detection of pass-containing windows across both splits.

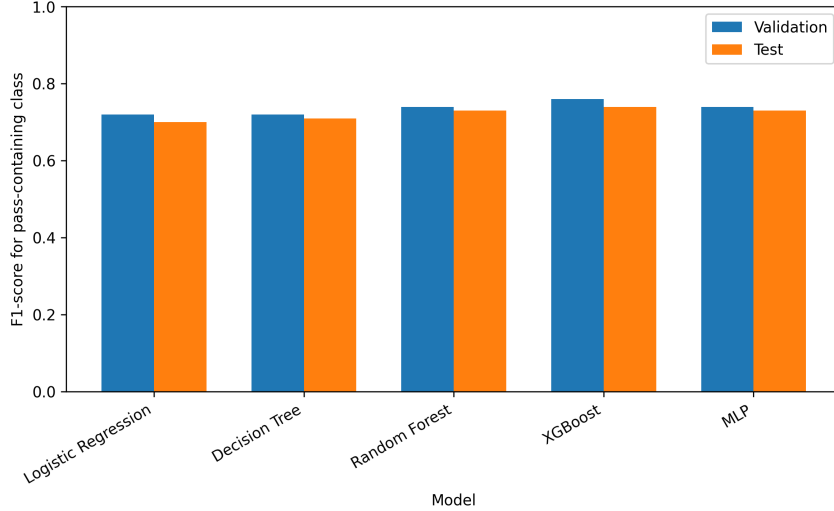


Figure 5: Comparison of validation and test F1-scores for the pass-containing class.

As shown in Table 7 in Appendix B.1, model complexity was also controlled through the selected hyperparameters. For Logistic Regression, the selected regularization parameter was relatively large, indicating weak regularization. For the tree-based models, parameters such as maximum depth, minimum samples per leaf, and minimum samples split limited how specific the learned decision rules could become. For example, the Decision Tree selected a maximum depth of 7, with 5 minimum samples per leaf and 2 minimum samples per split, while the Random Forest selected a maximum depth of 10, with 1 minimum sample per leaf and 2 minimum samples per split. For XGBoost, model complexity was controlled through a maximum depth of 5, together with regularization parameters and a subsampling rate of 0.8. Similarly, the MLP selected a relatively simple architecture with one hidden layer of 64 neurons, suggesting that a more complex neural network was not needed for this feature set.

However, overfitting cannot be completely ruled out. The models generally performed slightly better on the validation set than on the test set, which may indicate some degree of adaptation to the validation data during model development. This is particularly relevant because the validation set was used for threshold selection, meaning that the selected thresholds may be optimized for the specific validation matches.

Finally, the scope of generalization remains limited by the dataset itself. All data used in this study came from the same provider and from a similar match context. Therefore, the results should be interpreted as evidence of generalization to unseen matches within this dataset, rather than as proof that the models would generalize equally well to different leagues, age groups, tracking systems, or annotation procedures.

5.4 Limitations

The results show that pass windows can be detected from engineered positional features, but several limitations remain.

First, the quality and completeness of the tracking data affect the reliability of the extracted features, which is a known challenge in football tracking-data analysis [19, 3]. In some cases, positional data for the ball or players was missing. Although interpolation and NaN-based filtering were applied, missing values can still introduce inconsistencies in movement-based features such as ball speed, distance to the closest player, and changes in the closest player [16]. Since passes are short and dynamic events, even small tracking gaps can influence the quality of the extracted window-level features.

Second, the sampling rate limits the temporal detail available to the model. The data was sampled at 5 Hz, meaning that only five frames were available per second. While this captures general movement patterns, passes can occur quickly, and important changes in speed, direction, or player proximity may occur between frames. Higher-frequency tracking data, such as 10 Hz or 15 Hz, could improve the extraction of movement-based features and may lead to better pass detection performance.

Third, the dataset only contained two-dimensional x- and y-coordinates. Without a z-coordinate, the model cannot identify the height of the ball. This limits the ability to distinguish ground passes from aerial passes, clearances, crosses, or other forms of ball movement. Including ball-height information could improve the representation of different event types and make the extracted features more informative.

A further limitation is the window-based labeling strategy. In this study, 15 frame windows with a stride length of 15 were labeled as positive if they contained at least one pass. This allows the task to be formulated as binary classification, but it also means that multiple passes inside one window are represented by a single positive label. As a result, even a model with strong window-level performance may still miss individual event-level passes.

Another limitation concerns the manually selected thresholds used in some derived features. For example, features such as `player_changed_and_close_edges`, `player_changed_and_high_speed`, `high_speed_far_from_player_share`, and `high_speed_close_to_player_share` depend on fixed speed and distance thresholds. Ideally, thresholds such as the high-speed cut-off and player-ball distance cut-offs should be tuned systematically.

An additional limitation concerns the fixed window size and stride length. In this study, both were set to 15 frames, corresponding to non-overlapping 3 second windows. This provided a practical balance between temporal context, computational efficiency, and label interpretability. However, different window sizes or overlapping windows may produce different results. Shorter windows could improve temporal precision but may provide less context, while longer windows could capture broader movement patterns but may also include multiple actions within the same instance. Similarly, overlapping windows could improve event coverage but would increase the number of highly similar samples and require additional post-processing to avoid duplicate detections.

Finally, the current study only distinguishes between pass and no-pass windows. However, other football events, such as shots, clearances, interceptions, or tackles, may produce movement patterns similar to passes. This reflects a broader challenge in football event detection, where different event types can share similar spatiotemporal patterns [3]. Since

these events are included in the no-pass class, the model may confuse them with pass windows. A future multi-class event detection approach could help the model better separate passes from other types of ball actions.

These limitations highlight the difference between detecting pass-like movement patterns and fully identifying pass events. The results show that engineered positional features can provide useful predictive signals, but they also indicate that additional information and more refined temporal methods are needed to move from window-level classification towards reliable event-level pass detection.

5.5 Future Work

Future work could improve the approach by moving from window-level classification towards more precise temporal event detection. Frame-level prediction or sequence-labelling approaches could allow pass events to be localized more accurately, although this would require improved labelling strategies and stronger methods for handling class imbalance.

Another direction would be to develop more refined post-processing methods for converting positive windows into event-level detections. Since a single positive window can contain more than one pass, window-level labelling may compress multiple events into one positive instance. A secondary count classifier could be trained to estimate whether a positive window contains one pass or multiple passes, helping to approximate the number of events within each positive window. The proposed post-processing algorithm in Appendix C.1 provides one possible direction for converting window-level predictions into event-level detections.

Possession-based approaches could also be explored. Instead of relying only on movement patterns, passes could be identified through changes in ball proximity or possession between players on the same team, combined with ball speed, direction, and player-distance features. This may help distinguish passes from visually similar events such as clearances, shots, or interceptions.

Further improvements may also come from richer data sources. Higher-frequency tracking data could capture more detailed movement changes, while ball-height information could help distinguish ground passes from aerial events. Additional features describing team shape, pressure, passing lanes, player orientation, and tactical context may also improve the model’s ability to separate pass events from other types of ball movement.

Future work could also treat several feature engineering choices as tunable parameters rather than fixed design decisions. In this study, thresholds used in derived interaction features such as `player_changed_and_close_edges`, `player_changed_and_high_speed`, `high_speed_far_from_player_share`, and `high_speed_close_to_player_share` were selected manually based on domain intuition. Future experiments could systematically tune the speed and distance thresholds used in these features to determine whether alternative values improve model performance. Similarly, window size and stride length could be included as design parameters, allowing future work to evaluate how different temporal resolutions affect the trade-off between frame context, computational cost, and event-level interpretability.

From a practical perspective, this research can also support Forward Football’s ongoing move towards developing its own tracking system. Automatic pass detection could reduce the need for manual event annotation and provide a first step towards generating event

data directly from positional tracking data. Although the current approach does not yet produce complete event-level pass annotations, it shows that pass-containing moments can be identified from tracking data with relatively high recall. This makes the approach useful as a foundation for future internal tools, where predicted pass windows could be used to support analysts, guide manual review, or provide candidate events for a more refined detection pipeline.

Finally, future research could investigate more advanced temporal models, such as LSTMs or Transformer-based architectures. These models may be useful for learning sequential patterns directly from tracking data, especially if larger datasets, higher-frequency tracking data, or more detailed tracking information become available through Forward Football’s own tracking system.

6 Conclusion

This study investigated whether football passes can be predicted from raw positional tracking data using machine learning. The task was formulated as a window-level binary classification problem, where each 15 frame window was labelled according to whether it contained at least one pass. Logistic Regression, Decision Tree, Random Forest, XGBoost, and MLP models were trained and evaluated using engineered features derived from ball and player coordinates.

The results show that pass-containing windows can be detected from raw positional data when the coordinates are transformed into meaningful features. XGBoost achieved the strongest overall performance, followed closely by Random Forest and MLP, while Logistic Regression and Decision Tree performed less strongly. This suggests that non-linear models are better suited to the task, as pass prediction depends on interactions between ball movement, player proximity, changes in the closest player, and field position.

The feature importance results further show that raw coordinates alone are not the most informative representation of the data. Instead, features describing ball-player distance, changes in the closest player, ball movement path, direction, and pitch location contributed most strongly to model performance. These findings highlight the importance of feature engineering when applying machine learning to raw football tracking data.

The results should be interpreted as probabilistic window-level predictions rather than complete event-level annotations. The models estimate the likelihood that a window contains a pass, but they do not directly predict the exact timing or number of passes within that window. As a result, some events may still be missed or incorrectly predicted, particularly when pass-like movement patterns resemble other ball actions. Strong window-level performance therefore does not necessarily imply complete event-level pass detection.

This thesis can also support further research on football event detection. The preprocessing pipeline, window construction strategy, and engineered features provide a reusable starting point for future students or researchers working with similar tracking data. In particular, features based on ball movement, closest-player distance, player changes, and pitch location may be useful beyond pass detection and could inspire related work on other football events, such as receptions, clearances, interceptions, or shots. The study therefore contributes not only a pass-detection model, but also a structured methodology that can be adapted and extended in future research.

Overall, this study shows that raw positional data contains useful signals for pass prediction, but that these signals become most effective when transformed into engineered features that describe player-ball interactions and movement dynamics. The window-level approach provides a useful starting point for automatic pass detection and could support Forward Football’s development of future tracking-based event detection tools. At the same time, the methods developed in this thesis offer a foundation for further academic work on football event detection using positional tracking data.

References

- [1] ANZER, G., AND BAUER, P. Expected passes. *Data Mining and Knowledge Discovery* 36 (01 2022), 1–23.
- [2] BERGSTRA, J., AND BENGIO, Y. Random search for hyper-parameter optimization. *J. Mach. Learn. Res.* 13, null (Feb. 2012), 281–305.
- [3] BISCHOFBERGER, J., BACA, A., AND SCHIKUTA, E. Event detection in football: Improving the reliability of match analysis. *PLOS ONE* 19, 4 (2024), e0298107.
- [4] BOON, D. Expected successful passes. Unpublished report, Hogeschool van Amsterdam, 2021. Forward Football B.V.
- [5] BREIMAN, L. Random forests. *Machine Learning* 45 (10 2001), 5–32.
- [6] CHEN, T., AND GUESTRIN, C. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2016), pp. 785–794.
- [7] COX, D. R. The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B* 20, 2 (1958), 215–242.
- [8] EL KADI, T. Machine learning models for detecting line-breaking passes in football matches. Bachelor’s thesis, Vrije Universiteit Amsterdam, 2024.
- [9] GOODFELLOW, I., BENGIO, Y., AND COURVILLE, A. *Deep Learning*. MIT Press, 2016.
- [10] HARRIS, C. R., MILLMAN, K. J., VAN DER WALT, S. J., GOMMERS, R., VIRTANEN, P., COUNAPEAU, D., WIESER, E., TAYLOR, J., BERG, S., SMITH, N. J., KERN, R., PICUS, M., HOYER, S., VAN KERKWIJK, M. H., BRETT, M., HALDANE, A., DEL RÍO, J. F., WIEBE, M., PETERSON, P., GÉRARD-MARCHANT, P., SHEPPARD, K., REDDY, T., WECKESSER, W., ABBASI, H., GOHLKE, C., AND OLIPHANT, T. E. Array programming with numpy. *Nature* 585 (2020), 357–362.
- [11] HASSARD, P., AND KERR, D. Predicting football match outcomes using event data and machine learning algorithms. In *Proceedings of the 35th Irish Signals and Systems Conference (ISSC)* (06 2024), pp. 1–6.
- [12] HEROLD, M., GOES-SMIT, F., NOPP, S., BAUER, P., THOMPSON, C., AND MEYER, T. Machine learning in men’s professional football: Current applications and future directions for improving attacking play. *International Journal of Sports Science & Coaching* 14 (12 2019), 798–817.
- [13] HORTON, M., GUDMUNDSSON, J., CHAWLA, S., AND ESTEPHAN, J. Classification of passes in football matches using spatiotemporal data. *ACM Transactions on Spatial Algorithms and Systems* 3 (07 2014).
- [14] HUNTER, J. D. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering* 9, 3 (2007), 90–95.

- [15] KHAUSTOV, V., AND MOZGOVOY, M. Recognizing events in spatiotemporal soccer data. *Applied Sciences* 10 (11 2020), 8046.
- [16] KONTOS, C., AND KARLIS, D. Football analytics based on player tracking data using interpolation techniques for the prediction of missing coordinates. *Statistica Applicata – Italian Journal of Applied Statistics* 35, 2 (2023).
- [17] LIU, H., HOPKINS, W., GÓMEZ, A. M., AND MOLINUEVO, S. J. Inter-operator reliability of live football match statistics from opta sportsdata. *International Journal of Performance Analysis in Sport* 13, 3 (2013), 803–821.
- [18] MCKINNEY, W. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference* (2010), pp. 56–61.
- [19] MEMMERT, D., AND RAABE, D. *Data Analytics in Football: Positional Data Collection, Modelling and Analysis*, 2 ed. Routledge, 2023.
- [20] MORRA, L., MANIGRASSO, F., CANTO, G., GIANFRATE, C., GUARINO, E., AND LAMBERTI, F. Slicing and dicing soccer: Automatic detection of complex events from spatio-temporal data. In *Image Analysis and Recognition*. Springer, 2020, pp. 107–121.
- [21] PEDREGOSA, F., VAROQUAUX, G., GRAMFORT, A., MICHEL, V., THIRION, B., GRISEL, O., BLONDEL, M., PRETTENHOFER, P., WEISS, R., DUBOURG, V., VANDERPLAS, J., PASSOS, A., COURNAPEAU, D., BRUCHER, M., PERROT, M., AND DUCHESNAY, É. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research* 12, 85 (2011), 2825–2830.
- [22] QUINLAN, J. R. Induction of decision trees. *Machine Learning* 1 (1986), 81–106.
- [23] RESCH, A., PFEIL, R., WEGENER, M., AND STELZER, A. Review of the lpm local positioning measurement system. In *2012 International Conference on Localization and GNSS* (2012), IEEE, pp. 1–5.
- [24] RICHLI, K., BOTHE, M., ROHLOFF, T., AND SCHWARZ, C. Recognizing compound events in spatio-temporal football data. In *Proceedings of the 4th International Congress on Sport Sciences Research and Technology Support* (2016), pp. 27–35.
- [25] RICHLI, K., MORITZ, F., AND SCHWARZ, C. Utilizing artificial neural networks to detect compound events in spatio-temporal soccer data. In *Proceedings of the SIGKDD’17 Workshop on Mining and Learning from Time Series (MiLeTS)* (2017), pp. 1–7.
- [26] RUMELHART, D. E., HINTON, G. E., AND WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature* 323 (1986), 533–536.
- [27] SOFRONI, R.-Ş. Data-driven evaluation of player contributions during ball possession. Bachelor’s thesis, Vrije Universiteit Amsterdam, 2025.

- [28] TOVINKERE, V., AND QIAN, R. J. Detecting semantic events in soccer games: Towards a complete solution. In *Proceedings of the 2001 IEEE International Conference on Multimedia and Expo (ICME)* (2001), pp. 833–836.
- [29] VIDAL-CODINA, F., EVANS, N., FAKIR, B., AND BILLINGHAM, J. Automatic event detection in football using tracking data. *Sports Engineering* 25 (09 2022).
- [30] WASKOM, M. L. Seaborn: Statistical data visualization. *Journal of Open Source Software* 6, 60 (2021), 3021.
- [31] WINDT, J., TAYLOR, D., LITTLE, D., AND SPORER, B. C. Making everyone’s job easier: How do data scientists fit as a critical member of integrated support teams? *British Journal of Sports Medicine* 55, 2 (2021), 73–75.

A Features

A.1 Final Feature Set

Table 5: Overview of engineered features grouped by category

Category	Features
Position Features	x_center, y_center
Ball Displacement Features	ball_displacement_start_end, ball_displacement_from_pitch_origin
Ball Speed Features	speed_max, speed_mean, ball_speed_start, ball_speed_end, speed_end_minus_start, speed_prev_mean, speed_prev_std
Ball Direction Features	direction_max, direction_min, direction_mean
Distance to Closest Player Features	dist_max, dist_min, dist_mean, dist_center, dist_start, dist_end, dist_end_minus_start, dist_close_start, dist_close_end
Closest Player Dynamics	same_closest_player_all_window, closest_player_changes, closest_player_start_end_changed
Closest Team Dynamics	closest_team_changes, closest_team_start_end_changed
Ball-Player Proximity Features	ball_player_close_share, ball_player_close_any
Closest Player Streak Features	streak_max, streak_min, streak_mean, streak_start, streak_end_minus_start
Team Proximity Features	own_players_near_max, own_players_near_min, oppo_players_near_max, oppo_players_near_min
Player Change Distance Features	dist_player_change_mean
Ball Path Features	ball_path_length, ball_straight_distance, ball_curve_distance
Derived Interaction Features	speed_distance_interaction, player_changed_x_speed_max, player_changed_x_path_length, player_changed_and_close_edges, player_changed_and_high_speed, team_changed_x_speed_max, team_changed_x_path_length, high_speed_far_from_player_share, high_speed_close_to_player_share

A.2 Feature Importance

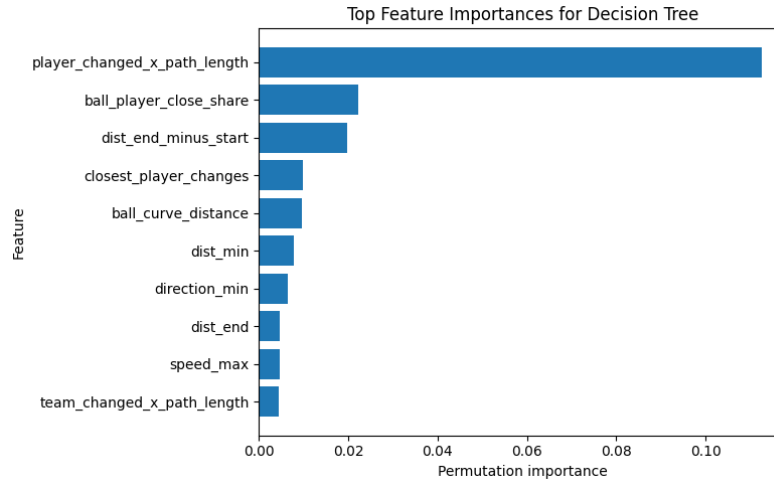


Figure 6: Top permutation importance scores for the Decision Tree model on the validation set

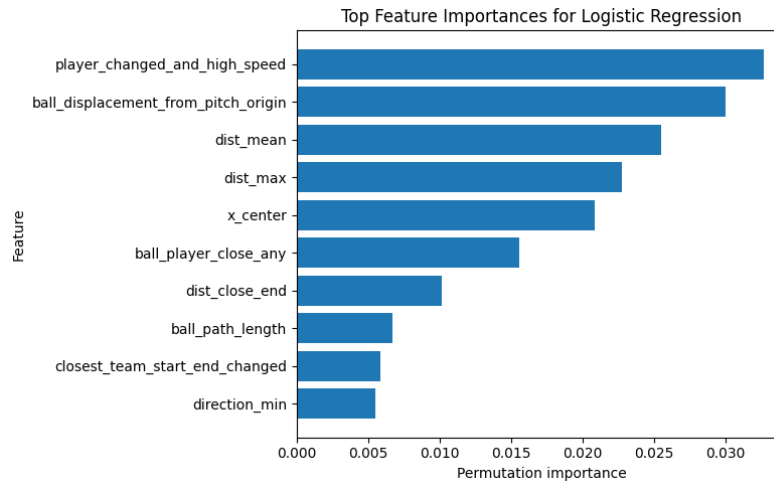


Figure 7: Top permutation importance scores for the Logistic Regression model on the validation set

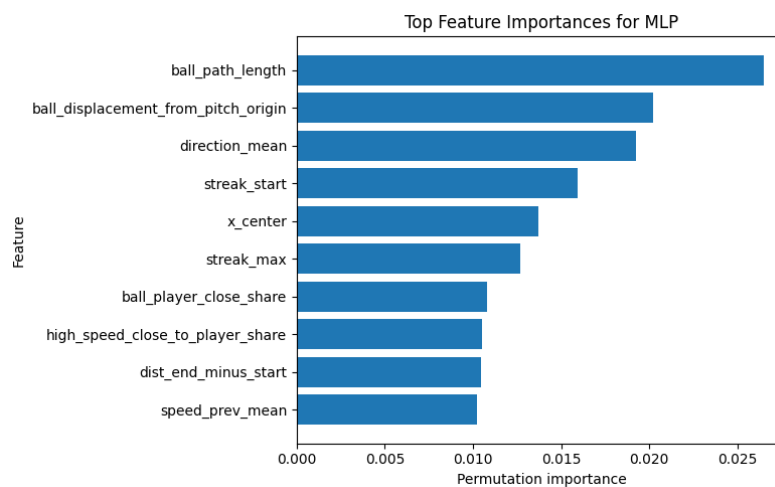


Figure 8: Top permutation importance scores for the MLP model on the validation set

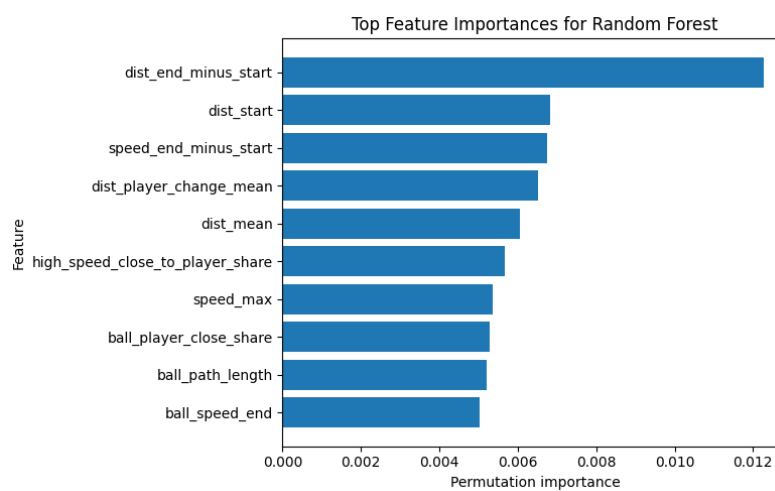


Figure 9: Top permutation importance scores for the Random Forest model on the validation set

B Hyperparameters

B.1 Hyperparameter Search Space

Table 6: Hyperparameter search space per model

Model	Hyperparameter	Values
Logistic Regression	Imputation	Mean, Median
	C	10^{-6} to 10^5
	Class weight	balanced; (0:1, 1: s); (0:1, 1:0.75); (0:1, 1:2); (0:1, 1:3)
Decision Tree	Imputation	Mean, Median
	Criterion	Gini, Entropy
	Max depth	3, 5, 7, None
	Min samples split	2, 10, 20
	Min samples leaf	1, 5, 10
	Class weight	balanced; (0:1, 1: s); (0:1, 1:0.75); (0:1, 1:2); (0:1, 1:3)
Random Forest	Imputation	Mean, Median
	Number of estimators	100, 300
	Criterion	Gini, Entropy
	Max depth	5, 7, 10
	Min samples split	2, 10
	Min samples leaf	1, 5
	Class weight	balanced; (0:1, 1: s); (0:1, 1:0.75); (0:1, 1:2); (0:1, 1:3)
XGBoost	Imputation	Mean, Median
	Number of estimators	100, 300
	Max depth	5, 7, 10
	Learning rate	0.01, 0.05, 0.1
	Subsample	0.8, 1.0
	Colsample by tree	0.8, 1.0
	Min child weight	1, 5
	Gamma	0, 0.3
	Regularization α	0, 0.1
	Regularization λ	1, 5
	Scale pos weight	0.75, 1, 2, 3, s
MLP	Imputation	Mean, Median
	Hidden layers	(32,), (64,), (64, 32)
	Activation	ReLU, Tanh
	Regularization (α)	0.0001, 0.001, 0.01
	Learning rate	0.0001, 0.001, 0.01
	Batch size	32, 64

B.2 Selected Hyperparameters

Table 7: Selected hyperparameters and classification thresholds per model

Model	Hyperparameter	Value
Logistic Regression		
	C	100000
	class_weight	(0:1, 1:s)
	imputer	median
	threshold	0.40
Decision Tree		
	max_depth	7
	min_samples_split	2
	min_samples_leaf	5
	criterion	gini
	class_weight	(0:1, 1:2)
	imputer	mean
	threshold	0.40
Random Forest		
	n_estimators	300
	max_depth	10
	min_samples_split	2
	min_samples_leaf	1
	criterion	entropy
	class_weight	balanced
	imputer	mean
	threshold	0.40
XGBoost		
	n_estimators	300
	max_depth	5
	learning_rate	0.05
	subsample	0.8
	colsample_bytree	1.0
	min_child_weight	5
	gamma	0
	reg_alpha	0.1
	reg_lambda	5
	scale_pos_weight	2
	imputer	mean
	threshold	0.45
MLP		
	hidden_layer_sizes	(64,)
	learning_rate_init	0.001
	batch_size	64
	alpha	0.001
	activation	relu
	imputer	mean
	threshold	0.30

C Postprocessing

C.1 Pass Compression

Table 8: Summary of pass compression caused by window-level labelling

Metric	Value
Positive windows	29,086
Real passes inside positive windows	33,824
Extra passes compressed into positive windows	4,738
Percentage of passes compressed	14.01%

C.2 Proposed Postprocessing Algorithm

Algorithm 1 Conceptual conversion of window-level predictions to event-level pass detections

Require: Window set W , binary pass classifier f_{pass} , count classifier f_{count}

Ensure: Set of detected pass events E

```
1:  $E \leftarrow \emptyset$ 
2: for each window  $w \in W$  do
3:    $\hat{y}_{\text{pass}} \leftarrow f_{\text{pass}}(w)$ 
4:    $\hat{c} \leftarrow f_{\text{count}}(w)$ 
5:   if  $\hat{y}_{\text{pass}} = 0$  then
6:      $n_{\text{events}} \leftarrow 0$ 
7:   else if  $\hat{y}_{\text{pass}} = 1$  and  $\hat{c} \in \{0, 1\}$  then
8:      $n_{\text{events}} \leftarrow 1$ 
9:   else if  $\hat{y}_{\text{pass}} = 1$  and  $\hat{c} = 2+$  then
10:     $n_{\text{events}} \leftarrow 2$ 
11:   end if
12:   if  $n_{\text{events}} = 1$  then
13:     Add detected pass event at the center frame of  $w$  to  $E$ 
14:   else if  $n_{\text{events}} = 2$  then
15:     Add detected pass event at one-third of  $w$  to  $E$ 
16:     Add detected pass event at two-thirds of  $w$  to  $E$ 
17:   end if
18: end for
19: return  $E$ 
```

D Validation Results

Table 9: Model performance on the validation set

Model	Prec (0)	Rec (0)	F1 (0)	Prec (1)	Rec (1)	F1 (1)
Logistic Regression	0.86	0.65	0.74	0.63	0.85	0.72
Decision Tree	0.87	0.64	0.73	0.63	0.86	0.72
Random Forest	0.87	0.68	0.76	0.65	0.85	0.74
XGBoost	0.88	0.70	0.78	0.67	0.87	0.76
MLP	0.86	0.69	0.77	0.66	0.85	0.74